

Microservices Architecture Documentation

1. Introduction

Dokumentasi ini dibuat untuk menjelaskan implementasi arsitektur microservices pada aplikasi Kelarin. Pada modul ini, backend monolith dipecah menjadi beberapa service independen agar sistem lebih modular, scalable, dan mudah dikelola.

Arsitektur microservices pada project ini terdiri dari:

- Auth Service
- Item Service
- API Gateway (Nginx)
- Database terpisah untuk setiap service
- Frontend Application

2. Architecture Overview

Microservices Architecture Diagram

```
graph TD
    User --> Frontend
    Frontend --> Gateway
    Gateway --> AuthService
    Gateway --> TaskService
    TaskService -->|Verify Token| AuthService
    AuthService --> AuthDB
    TaskService --> TaskDB
```

```
TaskService --> HealthAggregation
```

```
TaskService --> RetryLogic
```

```
TaskService --> CircuitBreaker
```

3. Reliability Features

Feature	Description
Health Aggregation	Menggabungkan status kesehatan service dan dependency untuk menentukan kondisi sistem secara keseluruhan.
Retry Logic	Melakukan percobaan ulang otomatis ketika komunikasi antar service gagal akibat timeout atau gangguan sementara.
Circuit Breaker	Mencegah service terus mengirim request ke dependency yang sedang bermasalah untuk menghindari cascading failure.
Health Monitoring	Monitoring kondisi service melalui endpoint <code>/health</code> dan Docker healthcheck.

4. Service Overview

Service	Port	Description
Frontend	80	User interface aplikasi
Gateway (Nginx)	80	API Gateway dan reverse proxy
Auth Service	8001	Authentication dan authorization service
Item Service	8002	CRUD item management
Auth Database	5432	Database khusus authentication
Item Database	5432	Database khusus item service

5. API Contract

Auth Service

Method	Endpoint	Description
POST	<code>/register</code>	Register user baru
POST	<code>/login</code>	Login user

Method	Endpoint	Description
GET	<code>/verify</code>	Verifikasi JWT token
GET	<code>/health</code>	Healthcheck auth-service

Task Service

Method	Endpoint	Description
GET	<code>/tasks</code>	Menampilkan seluruh task
GET	<code>/tasks/{id}</code>	Menampilkan detail task
POST	<code>/tasks</code>	Menambahkan task baru
PUT	<code>/tasks/{id}</code>	Mengubah task
DELETE	<code>/tasks/{id}</code>	Menghapus task
GET	<code>/health</code>	Healthcheck task-service

6. Docker Compose Workflow

Command	Description
<code>docker compose up --build</code>	Menjalankan seluruh container microservices
<code>docker compose down</code>	Menghentikan seluruh container
<code>docker compose ps</code>	Melihat status container
<code>docker compose logs auth-service</code>	Melihat logs auth-service
<code>docker compose logs item-service</code>	Melihat logs item-service

7. Local Development Setup

Prerequisites

- Docker Desktop
- Docker Compose
- Git

Menjalankan Seluruh Service

```
docker compose up --build
```

8. Verifikasi Status Service

```
docker compose ps
```

Expected Result:

- auth-service → healthy
- task-service → healthy
- auth-db → healthy
- task-db → healthy
- gateway-nginx → healthy
- frontend → healthy

Menghentikan Service

```
docker compose down
```

9. Healthcheck & Monitoring

Health Endpoint

Service	Endpoint	Status
Auth Service	/health	☒ Active
Item Service	/health	☒ Active

Monitoring Logs

Command	Function
<code>docker compose logs auth-service</code>	Monitoring auth-service
<code>docker compose logs item-service</code>	Monitoring item-service

10. Testing Validation

Docker Compose Validation

Service	Status
auth-service	☒ Healthy
item-service	☒ Healthy
auth-db	☒ Healthy
item-db	☒ Healthy

Service	Status
frontend	☑ Healthy
gateway-nginx	☑ Healthy

Keterangan

- Seluruh container berhasil dijalankan menggunakan Docker Compose.
- Healthcheck service menunjukkan status healthy.
- Tidak ditemukan crash pada service utama.

11. Health Endpoint Testing

Test Case	Result
Auth Service /health accessible	☑ Passed
Item Service /health accessible	☑ Passed
Health status returns 200 OK	☑ Passed

Keterangan

- Endpoint health berhasil diakses melalui Swagger dan Docker healthcheck.
- Service memberikan response **200 OK**.
- Monitoring service berjalan dengan normal tanpa error.

12. Feature Testing

Feature	Result
Register User	☑ Passed
Login User	☑ Passed
Create Task	☑ Passed
Read Task	☑ Passed
Update Task	☑ Passed
Delete Task	☑ Passed

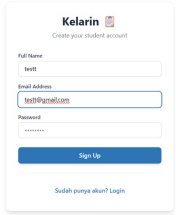
Keterangan

- Seluruh fitur utama aplikasi berhasil berjalan pada implementasi microservices.
- Frontend berhasil terhubung dengan API Gateway dan backend services.

13. Development vs Microservices Comparison

Component	Monolith Architecture	Microservices Architecture
Backend Structure	Single backend	Multiple independent services
Database	Single database	Database per service
Deployment	Single container	Multi-container
Scalability	Limited	More scalable
Fault Isolation	Single point of failure	Service isolation

14. Documentation Evidence

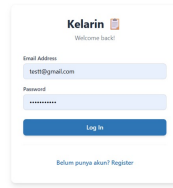
Testing Activity	Evidence	Description
Docker Compose Validation	<pre>(base) PS C:\Users\DELL 4510\cc-kelompok-a-ethereal> docker compose ps NAME COMMAND STATUS PORTS backend delia/kelarin-backend-v2 Up 2 hours 8080->8080/tcp db-kelarin postgresql@15-alpine Up 2 hours 5432->5432/tcp frontend delia/kelarin-frontend-v1 Up 2 hours 5177->5177/tcp</pre>	Pengujian dilakukan menggunakan perintah docker compose ps untuk memastikan seluruh container microservices berjalan dengan status running dan healthy.
Auth Service Healthcheck	<pre>(base) PS C:\Users\DELL 4510\cc-kelompok-a-ethereal> docker compose logs auth-service auth-service INFO: 127.0.0.1:37874 - "GET /health HTTP/1.1" 200 OK auth-service INFO: 127.0.0.1:44890 - "GET /health HTTP/1.1" 200 OK auth-service INFO: 127.0.0.1:43076 - "GET /health HTTP/1.1" 200 OK auth-service INFO: 127.0.0.1:53478 - "GET /health HTTP/1.1" 200 OK auth-service INFO: 127.0.0.1:40536 - "GET /health HTTP/1.1" 200 OK auth-service INFO: 127.0.0.1:53346 - "GET /health HTTP/1.1" 200 OK auth-service INFO: 127.0.0.1:37816 - "GET /health HTTP/1.1" 200 OK auth-service INFO: 127.0.0.1:52392 - "GET /health HTTP/1.1" 200 OK auth-service INFO: 127.0.0.1:33108 - "GET /health HTTP/1.1" 200 OK auth-service INFO: 127.0.0.1:33842 - "GET /health HTTP/1.1" 200 OK auth-service INFO: 127.0.0.1:39268 - "GET /health HTTP/1.1" 200 OK auth-service INFO: 127.0.0.1:44996 - "GET /health HTTP/1.1" 200 OK auth-service INFO: 127.0.0.1:41498 - "GET /health HTTP/1.1" 200 OK auth-service INFO: 127.0.0.1:58068 - "GET /health HTTP/1.1" 200 OK auth-service INFO: 127.0.0.1:32836 - "GET /health HTTP/1.1" 200 OK auth-service INFO: 127.0.0.1:49588 - "GET /health HTTP/1.1" 200 OK</pre>	Pengujian dilakukan pada endpoint /health auth-service melalui Swagger untuk memastikan service authentication berjalan normal dengan response 200 OK .
Item Service Healthcheck	<pre>(base) PS C:\Users\DELL 4510\cc-kelompok-a-ethereal> docker compose logs item-service item-service INFO: 127.0.0.1:53704 - "GET /health HTTP/1.1" 200 OK item-service INFO: 127.0.0.1:58378 - "GET /health HTTP/1.1" 200 OK item-service INFO: 127.0.0.1:37868 - "GET /health HTTP/1.1" 200 OK item-service INFO: 127.0.0.1:50884 - "GET /health HTTP/1.1" 200 OK item-service INFO: 127.0.0.1:48380 - "GET /health HTTP/1.1" 200 OK item-service INFO: 127.0.0.1:52548 - "GET /health HTTP/1.1" 200 OK item-service INFO: 127.0.0.1:35114 - "GET /health HTTP/1.1" 200 OK item-service INFO: 127.0.0.1:53528 - "GET /health HTTP/1.1" 200 OK item-service INFO: 127.0.0.1:32978 - "GET /health HTTP/1.1" 200 OK item-service INFO: 127.0.0.1:39716 - "GET /health HTTP/1.1" 200 OK item-service INFO: 127.0.0.1:58206 - "GET /health HTTP/1.1" 200 OK item-service INFO: 127.0.0.1:50874 - "GET /health HTTP/1.1" 200 OK item-service INFO: 127.0.0.1:44788 - "GET /health HTTP/1.1" 200 OK item-service INFO: 127.0.0.1:34696 - "GET /health HTTP/1.1" 200 OK item-service INFO: 127.0.0.1:38276 - "GET /health HTTP/1.1" 200 OK item-service INFO: 127.0.0.1:53812 - "GET /health HTTP/1.1" 200 OK</pre>	Pengujian dilakukan pada endpoint /health item-service untuk memastikan service item management berjalan normal tanpa error.
User Registration Testing		Pengujian dimulai dengan membuat akun baru melalui halaman registrasi untuk memastikan auth-service dapat menyimpan data user dengan baik.

Testing Activity

Evidence

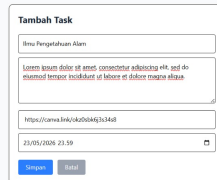
Description

User Login Testing



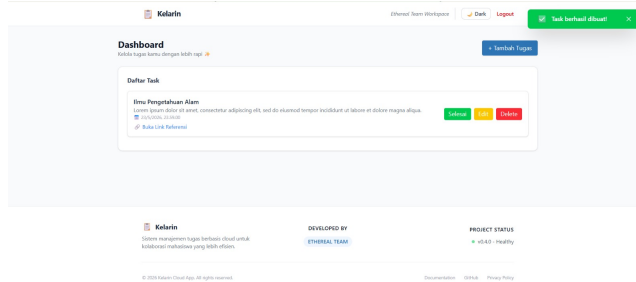
Pengujian dilakukan menggunakan akun yang telah terdaftar untuk memastikan proses authentication dan JWT token berjalan normal.

Create Item Testing



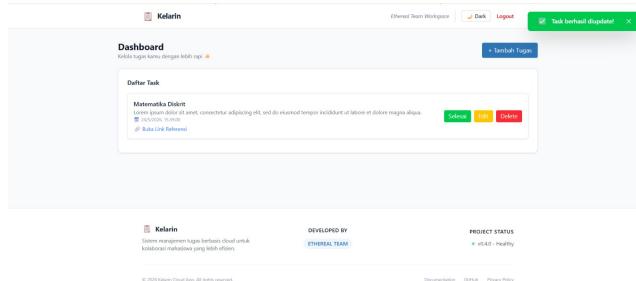
Pengujian dilakukan dengan menambahkan item baru pada aplikasi untuk memastikan item-service dapat menerima dan menyimpan data dengan baik.

Read Item Testing



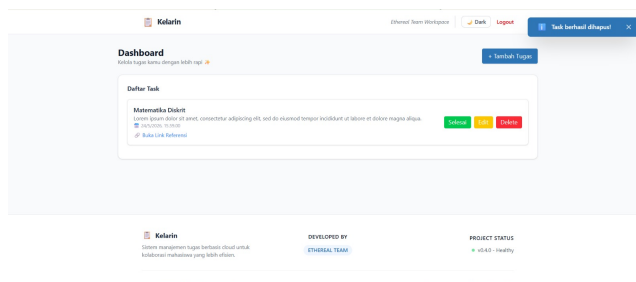
Pengujian dilakukan untuk memastikan item yang telah dibuat berhasil ditampilkan pada frontend application.

Update Item Testing

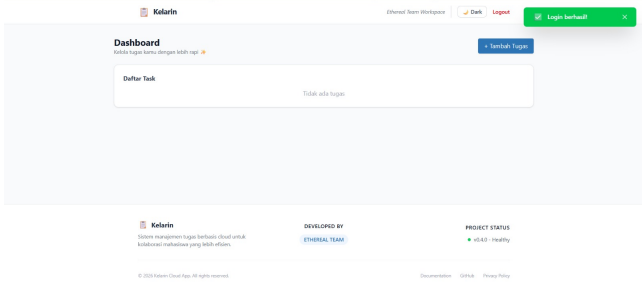


Pengujian dilakukan dengan mengubah data item untuk memastikan fitur update berjalan normal pada microservices architecture.

Delete Item Testing



Pengujian dilakukan dengan menghapus item dari sistem untuk memastikan delete endpoint berjalan dengan baik tanpa error.

Testing Activity	Evidence	Description
Frontend Microservices Testing		Pengujian dilakukan dengan membuka aplikasi melalui gateway nginx untuk memastikan frontend berhasil terhubung dengan backend microservices.

15. Service Debug Guide

Auth Service

Melihat log Auth Service:

```
docker compose logs auth-service
```

Health Endpoint:

```
http://localhost:8001/health
```

Task Service

Melihat log Task Service:

```
docker compose logs task-service
```

Health Endpoint:

```
http://localhost:8002/health
```

API Gateway

Melihat log Gateway:

```
docker compose logs gateway
```


Endpoint:

```
http://localhost
```

Common Troubleshooting

Problem	Solution
Container tidak berjalan	Jalankan <code>docker compose up --build</code>
Service unhealthy	Cek logs menggunakan <code>docker compose logs <service-name></code>
Frontend tidak dapat connect backend	Periksa konfigurasi gateway nginx
Endpoint tidak dapat diakses	Pastikan container running dan network Docker aktif
Healthcheck gagal	Periksa dependency service dan database
Gateway tidak meneruskan request	Periksa konfigurasi reverse proxy nginx

13. Conclusion

Implementasi microservices pada aplikasi Kelarin berhasil dijalankan menggunakan Docker Compose dengan arsitektur multi-service.

Seluruh service:

- berhasil berjalan
- saling terhubung
- memiliki healthcheck
- dapat melakukan komunikasi antar service dengan normal

Pengujian fitur utama aplikasi menunjukkan bahwa sistem berjalan stabil pada implementasi microservices.

Final Status

🟢 MICROSERVICES ARCHITECTURE RUNNING SUCCESSFULLY